

Movie Ratings API

The movie discovery platform needs a robust error handling layer. Right now the API returns ugly 500 stack traces when something goes wrong — the frontend team cannot work with that. All endpoints and services are already built and working. Your job is to implement the `GlobalExceptionHandler` so every failure case returns a clean, consistent JSON response.

What's already provided:

- `Movie.java` and `Rating.java` — models with validation annotations
- `MovieRepository.java` and `RatingRepository.java` — JPA repositories, ready to use
- `MovieService.java` and `RatingService.java` — fully implemented service layer
- `MovieController.java` and `RatingController.java` — fully implemented endpoints
- `MovieNotFoundException.java` and `DuplicateRatingException.java` — custom exceptions, already being thrown by the services
- `ErrorResponse.java` — response wrapper with `status` and `message` fields, ready to use
- `GlobalExceptionHandler.java` — class exists with `@RestControllerAdvice`, handlers are empty — this is where you work
- `DataLoader.java` — seeds movies and a duplicate reviewer scenario on startup so you can test without setting anything up

Specifically, you are expected to:

1. Observe the problem first — hit a missing movie ID and see the raw 500 stack trace so you understand what you are fixing.
2. Handle 404 by implementing a handler for `MovieNotFoundException` that returns a structured JSON response with HTTP 404.
3. Handle 400 by implementing a handler for `MethodArgumentNotValidException` that returns the first validation error message with HTTP 400.

4. Handle 409 by implementing a handler for `DuplicateRatingException` that returns a structured JSON response with HTTP 409 Conflict.
5. Handle any other unexpected exception with a generic fallback handler that returns HTTP 500 with a safe message instead of exposing a stack trace.
6. Add a `MovieAlreadyExistsException`, throw it in the service when a movie with the same title is added twice, and handle it in the global handler with HTTP 409.

WHY GRACEFUL HANDLING IS NEEDED?

When Spring encounters an unhandled exception it exposes a raw stack trace or its own default error format — both are problematic in production. A stack trace leaks internal implementation details like class names, method names, line numbers, and library versions to whoever is calling the API, which is a serious security vulnerability because attackers can use that information to craft targeted exploits. Spring's default error format on the other hand returns inconsistent, verbose JSON that your frontend team cannot reliably parse — it changes shape depending on the error type, includes fields your clients don't need, and gives no guarantee of structure. A

`GlobalExceptionHandler` solves both problems at once — every error, regardless of where it's thrown in the application, returns the same predictable `{"status": ..., "message": "..."}` shape that the frontend can handle uniformly, while keeping your internals completely hidden from the outside world.

Deliverables:

- `GET /api/movies/9999` returns `{"status": 404, "message": "..."}` not a stack trace.
- `POST /api/movies` with a blank title returns `{"status": 400, "message": "..."}` .
- `POST /api/movies/1/ratings` with a duplicate reviewer returns `{"status": 409, "message": "..."}` .

- Any unhandled exception returns {"status": 500, "message": "An unexpected error occurred"}.
- Stretch — adding a duplicate movie title returns {"status": 409, "message": "..."}.

Testing Endpoints

Observe the problem — before implementing any handler, run this and see the raw stack trace:

```
curl -s http://localhost:8080/api/movies/9999 | cat
```

After Task 2 — same request should now return clean JSON:

```
curl -s http://localhost:8080/api/movies/9999 | cat
```

Add a movie with blank title — expect 400:

```
curl -s -X POST http://localhost:8080/api/movies \
-H "Content-Type: application/json" \
-d '{"title": "", "genre": "sci-fi"}' | cat
```

Rate with invalid stars — expect 400:

```
curl -s -X POST http://localhost:8080/api/movies/1/ratings \
-H "Content-Type: application/json" \
-d '{"stars": 9, "reviewer": "Charlie"}' | cat
```

Rate a non-existent movie — expect 404:

```
curl -s -X POST http://localhost:8080/api/movies/9999/ratings \
-H "Content-Type: application/json" \
-d '{"stars": 3, "reviewer": "Dave"}' | cat
```

Duplicate reviewer — expect 409 (Alice is already seeded in DataLoader):

```
curl -s -X POST http://localhost:8080/api/movies/1/ratings \
-H "Content-Type: application/json" \
-d '{"stars": 3, "reviewer": "Alice"}' | cat
```

Add a movie:

```
curl -s -X POST http://localhost:8080/api/movies \  
-H "Content-Type: application/json" \  
-d '{"title": "The Matrix", "genre": "sci-fi"}' | cat
```

Get all movies:

```
curl -s http://localhost:8080/api/movies | cat
```

Get all ratings for a movie:

```
curl -s http://localhost:8080/api/movies/1/ratings | cat
```

Add same movie title twice — expect 409:

```
curl -s -X POST http://localhost:8080/api/movies \  
-H "Content-Type: application/json" \  
-d '{"title": "Inception", "genre": "sci-fi"}' | cat
```

Average rating for a movie:

```
curl -s http://localhost:8080/api/movies/1/ratings/average | cat
```

Average for movie with no ratings:

```
curl -s http://localhost:8080/api/movies/3/ratings/average | cat
```